Lesson 11 Teacher's Key: Wave After Wave

TIC-80 - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

The plainest possible version of this week's idea: one more function (spawn_wave()), one more nested if - built from the exact same three Lua primitives (functions, variables, conditionals) TIC-80 has used all along.

Level 1 - Full Worked Solution

- spawn_wave() replaces the table-populating portion of start_game() - a clean split between 'reset everything' (start_game) and 'create a wave's worth of enemies' (spawn_wave).
- wave and total_waves are new globals, parallel to lives and score.
- The bullets loop's hit branch grows a nested if - the same two-part decision every other engine builds this week.

Why This Structure

- TIC-80 has no special 'wave system' feature to reach for - this lesson, like Lesson 10's restart flow, is built entirely from ordinary Lua tools, which is a good, honest illustration of how far a few simple primitives can go.
- Replacing the enemies table with a fresh one (enemies={}) inside spawn_wave(), rather than trying to modify the old (now-empty) one in place, avoids any risk of leftover stale entries between waves.

Level 2 / Level 3 Talking Points

- Ask students to trace what 'enemies={}' actually does each time spawn_wave() runs - it doesn't just clear old entries, it replaces the whole table with a brand new empty one, which is a subtly different (and safer) operation worth understanding.
- For advanced students: point out that hard-coding 5 enemies per wave is a deliberate simplification - ask them to imagine a table like waves={5,7,9} indexed by the current wave, as a preview of Lesson 14's difficulty scaling.

Common Student Mistakes

- Checking only 'enemies_remaining<=0' without the wave comparison.
- Calling spawn_wave() inside the win branch (game_active=false case) by mistake.

- Forgetting wave=1 inside start_game(), leaving it at whatever value it had before a restart.